САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Свинин Артем

Группа К3342

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

1. Что было сделано

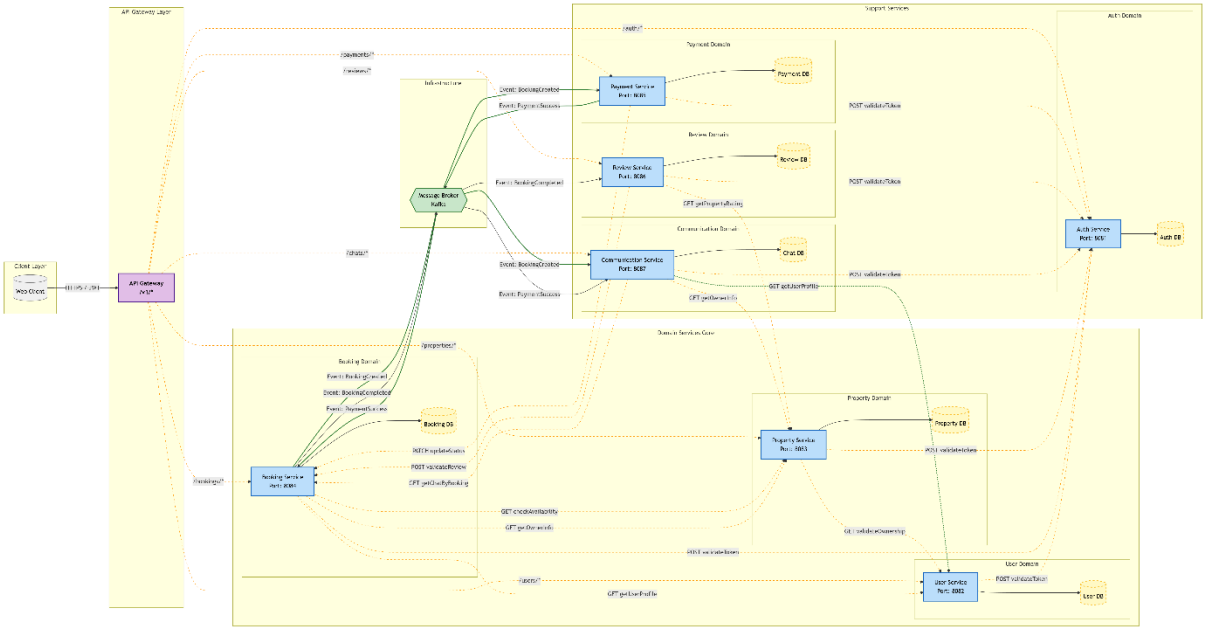
В рамках данного задания была спроектирована микросервисная архитектура для платформы аренды недвижимости. Монолитное приложение разделено на 7 независимых сервисов, каждый из которых отвечает за свой домен и имеет собственную базу данных. Разработаны две OpenAPI–спецификации: внешнее API для клиентов и внутреннее API для межсервисного взаимодействия.

2. Архитектура системы

2.1. Общая схема

Система теперь состоит из пяти уровней: клиентский слой, API Gateway, основные доменные сервисы, вспомогательные сервисы и инфраструктурный слой с брокером сообщений.

На диаграмме показана полная схема архитектуры. Сплошные стрелки обозначают синхронные HTTP–вызовы, пунктирные оранжевые – внутренние синхронные вызовы между сервисами, зелёные жирные – асинхронные события через Kafka.



2.2. Описание слоёв архитектуры

Клиентский слой представлен веб–приложением, которое взаимодействует с бэкендом исключительно через HTTPS с JWT–токеном в заголовке Authorization.

API Gateway – единая точка входа для всех клиентских запросов. Он принимает запрос, проверяет токен через Auth Service и перенаправляет запрос нужному сервису в зависимости от пути.

Основные доменные сервисы – User Service, Property Service и Booking Service – реализуют ключевую бизнес–логику платформы. Вспомогательные сервисы – Auth Service, Payment Service, Review Service и Communication Service – обеспечивают поддерживающие функции.

Инфраструктурный слой включает Apache Kafka, через который сервисы обмениваются асинхронными событиями, не зная друг о друге напрямую.

2.3. Перечень сервисов

Сервис	Зона ответственности
Auth Service	Аутентификация, выдача и валидация JWT–токенов, управление сессиями
User Service	Профили пользователей, список избранного
Property Service	Объявления о недвижимости, фотографии, поиск и фильтрация
Booking Service	Бронирования, проверка доступности дат, смена статусов
Payment Service	Платежи, транзакции, возвраты средств
Review Service	Отзывы и рейтинги объектов, владельцев и гостей
Communication Service	Чаты и сообщения между арендатором и владельцем

3. Разделение баз данных

Каждый сервис работает со своей отдельной базой данных по Database–per–Service. Никакой сервис не имеет прямого доступа к чужой базе данных. Если одному сервису нужны данные из другого, он запрашивает их через API.

Auth Service хранит учётные данные пользователей (email, хеш пароля) и refresh–токены. User Service хранит публичный профиль – имя, фото, телефон – и список избранных объектов. Property Service хранит объявления, адреса, изображения и удобства. Booking Service хранит только бронирования и даты, ссылаясь на объекты и пользователей по их идентификаторам. Payment Service хранит историю транзакций. Review Service – отзывы, привязанные к бронированиям. Communication Service – чаты и сообщения.

4. Описание диаграммы взаимодействий

4.1. Как читать диаграмму

На схеме (Рис. 1) используются три типа связей:

- Сплошные чёрные стрелки – синхронные вызовы от клиента через Gateway к сервисам по HTTPS.
- Оранжевые пунктирные стрелки – синхронные внутренние вызовы между сервисами по HTTP (используют внутренний API с сервисным токеном).
- Зелёные жирные стрелки – асинхронные события через Apache Kafka.

4.2. Синхронные внутренние вызовы

Когда пользователь создаёт бронирование, Booking Service обращается к Property Service, чтобы проверить доступность выбранных дат и получить данные о владельце. Параллельно он обращается к User Service, чтобы убедиться, что арендатор существует и активен. Все эти вызовы происходят синхронно – ответ клиенту не отправляется, пока все проверки не пройдены.

Каждый сервис при получении входящего запроса проверяет JWT–токен пользователя через Auth Service. Это единственный компонент, к которому обращаются абсолютно все остальные сервисы.

Property Service при создании или редактировании объекта обращается к User Service, чтобы убедиться, что пользователь действительно является владельцем этого объекта.

4.3. Асинхронные события через Kafka

Асинхронная коммуникация используется там, где результат не нужен немедленно, а важна надёжность доставки. Когда создаётся бронирование, Booking Service публикует событие BookingCreated в Kafka. Payment Service получает это событие и инициирует платёж. Communication Service получает то же событие и автоматически создаёт чат между арендатором и владельцем. Таким образом, Booking Service не знает и не зависит от того, есть ли вообще Payment Service или Communication Service в системе.

После успешной оплаты Payment Service публикует событие PaymentSuccess. Booking Service подписан на него и переводит бронирование в статус CONFIRMED. Communication Service также получает это событие и отправляет системное сообщение в чат об успешной оплате.

Когда бронирование завершается, Booking Service публикует событие BookingCompleted. Review Service подписан на него и открывает

возможность оставить отзыв – без этого события написать отзыв невозможно.

Событие	Кто публикует	Кто получает и что делает
BookingCreated	Booking Service	Payment Service – создаёт транзакцию; Communication Service – создаёт чат
PaymentSuccess	Payment Service	Booking Service – статус CONFIRMED; Communication Service – системное сообщение
PaymentFailed	Payment Service	Booking Service – статус CANCELLED
BookingCompleted	Booking Service	Review Service – открывает возможность оставить отзыв
BookingCancelled	Booking Service	Payment Service – инициирует возврат средств

5. API–спецификации

5.1. Внешнее API (openapi.yaml)

Внешнее API описывает все эндпоинты, доступные клиентскому приложению через Gateway. Спецификация включает 7 групп эндпоинтов по числу сервисов.

Аутентификация пользователя реализована через POST /auth/login – в ответ возвращается пара токенов access и refresh. Access–токен передаётся в заголовке Authorization: Bearer при каждом последующем запросе. Обновление токенов без повторного входа реализовано через POST /auth/refresh.

Поиск объектов недвижимости доступен через GET /properties с параметрами фильтрации: город, диапазон цен, даты заезда и выезда, тип

объекта. Результаты возвращаются постранично. Перед созданием бронирования клиент может проверить доступность конкретного объекта на выбранные даты через GET /properties/{id}/availability.

Создание бронирования – POST /bookings – запускает цепочку событий: платёж, создание чата, а после завершения – возможность оставить отзыв. Все изменения статусов бронирования (подтверждение, отмена, отклонение) реализованы отдельными PATCH–эндпоинтами.

5.2. Внутреннее API (openapi_internal.yaml)

Внутреннее API предназначено исключительно для межсервисного взаимодействия и недоступно снаружи Gateway. Все эндпоинты защищены сервисным токеном, передаваемым в заголовке X–Service–Token. Базовый путь – /internal/*.

Auth Service предоставляет эндпоинт POST /internal/auth/tokens/validate – единственный способ, которым другие сервисы проверяют токен пользователя. В ответ возвращается идентификатор пользователя, его роль и статус активности. Дополнительно реализована batch–валидация сразу нескольких пользователей через POST /internal/auth/users/validate – это используется при загрузке списков, чтобы не делать по одному запросу на каждого пользователя.

Booking Service предоставляет для Payment Service эндпоинт PATCH /internal/bookings/{id}/status – именно через него платёжный сервис сообщает об успехе или неудаче оплаты. Property Service предоставляет для Booking Service эндпоинт GET /internal/properties/{id}/availability с подробным ответом: доступны ли даты, если нет – какие конкретно бронирования конфликтуют, а также итоговая стоимость аренды.

5.3. Документирование ошибок

Код	Когда возникает
400	Некорректные данные в запросе (отсутствующее поле, неверный формат)
401	Токен отсутствует, истёк или недействителен
403	Попытка выполнить действие над чужим ресурсом
404	Запрашиваемый ресурс не найден
409	Конфликт: дублирующийся email, даты пересекаются, объект уже в избранном
429	Превышен лимит попыток входа
500	Внутренняя ошибка сервера
503	Целевой сервис временно недоступен (внутреннее API)